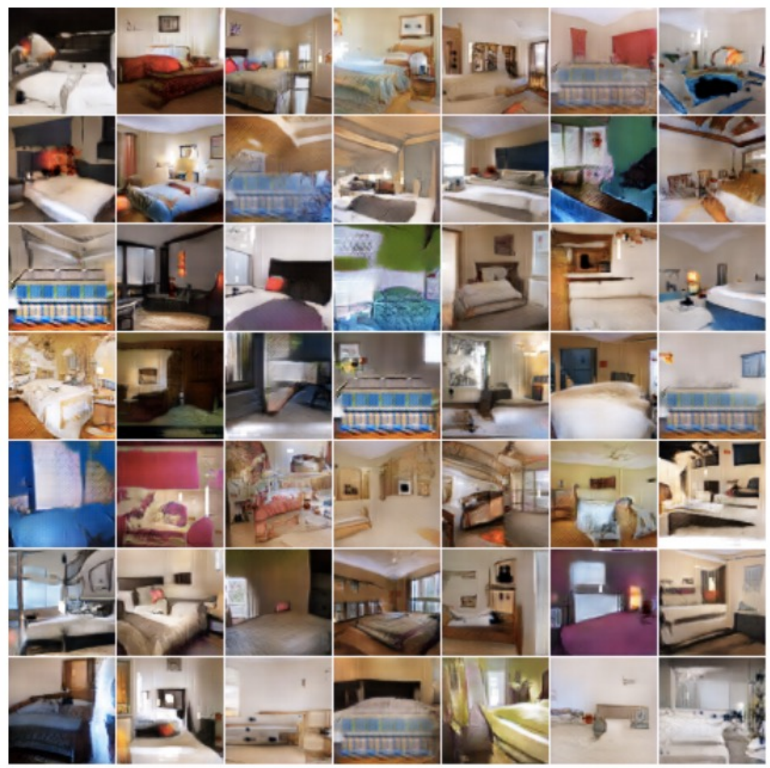
Generative image modeling: Diffusion models

Generative modeling tasks

- Generation: learn to sample from the distribution represented by the training set



Generative modeling tasks

- Generation conditioned on class label or text prompt

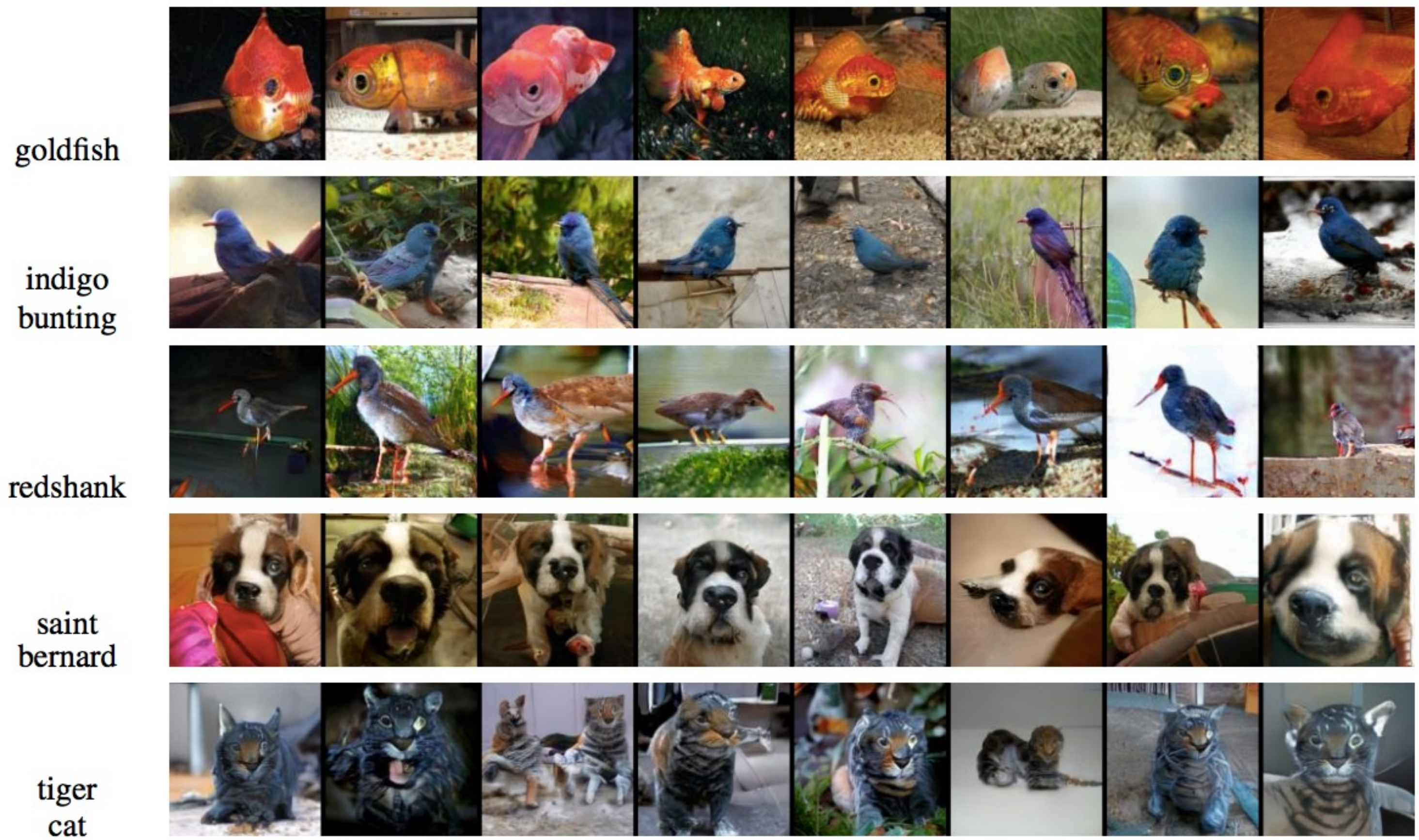
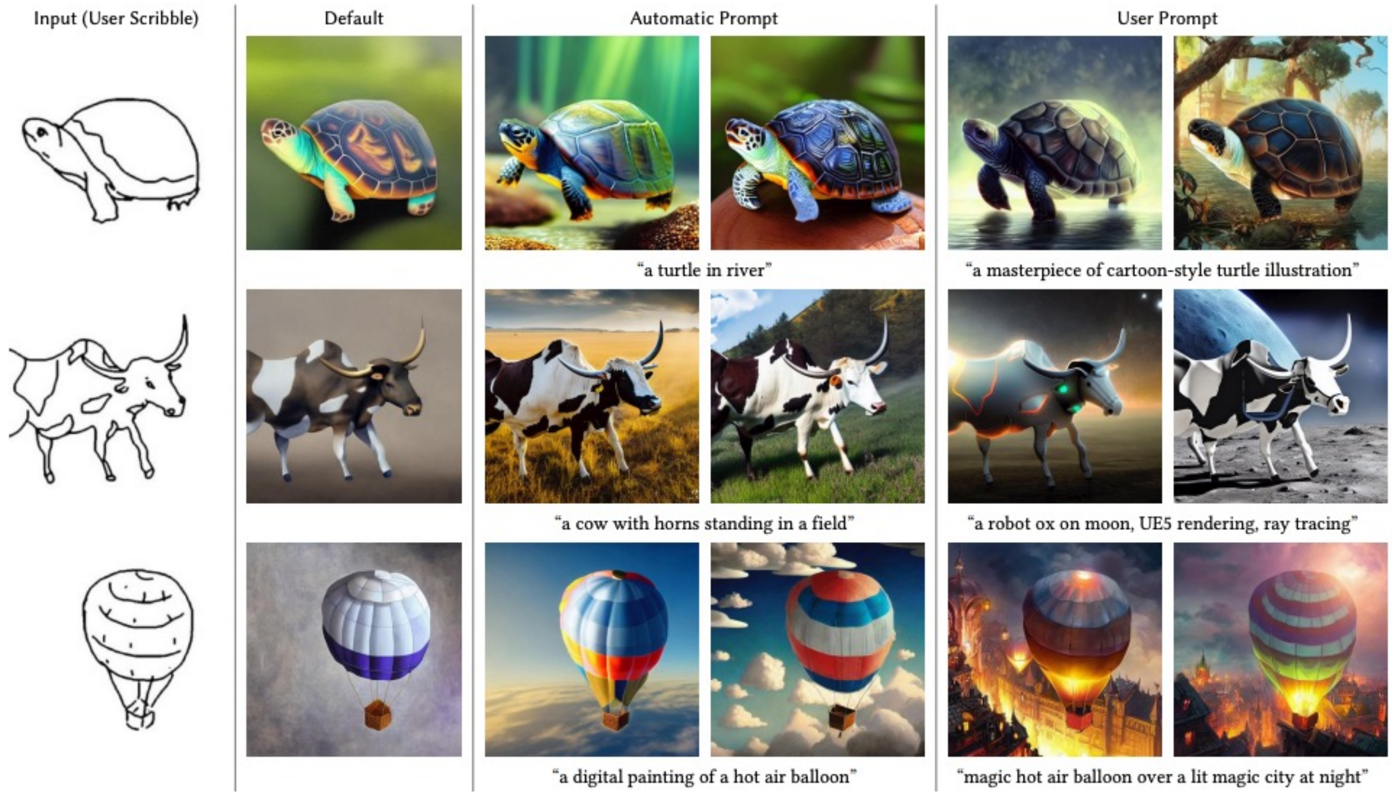


Figure source

Generative modeling tasks

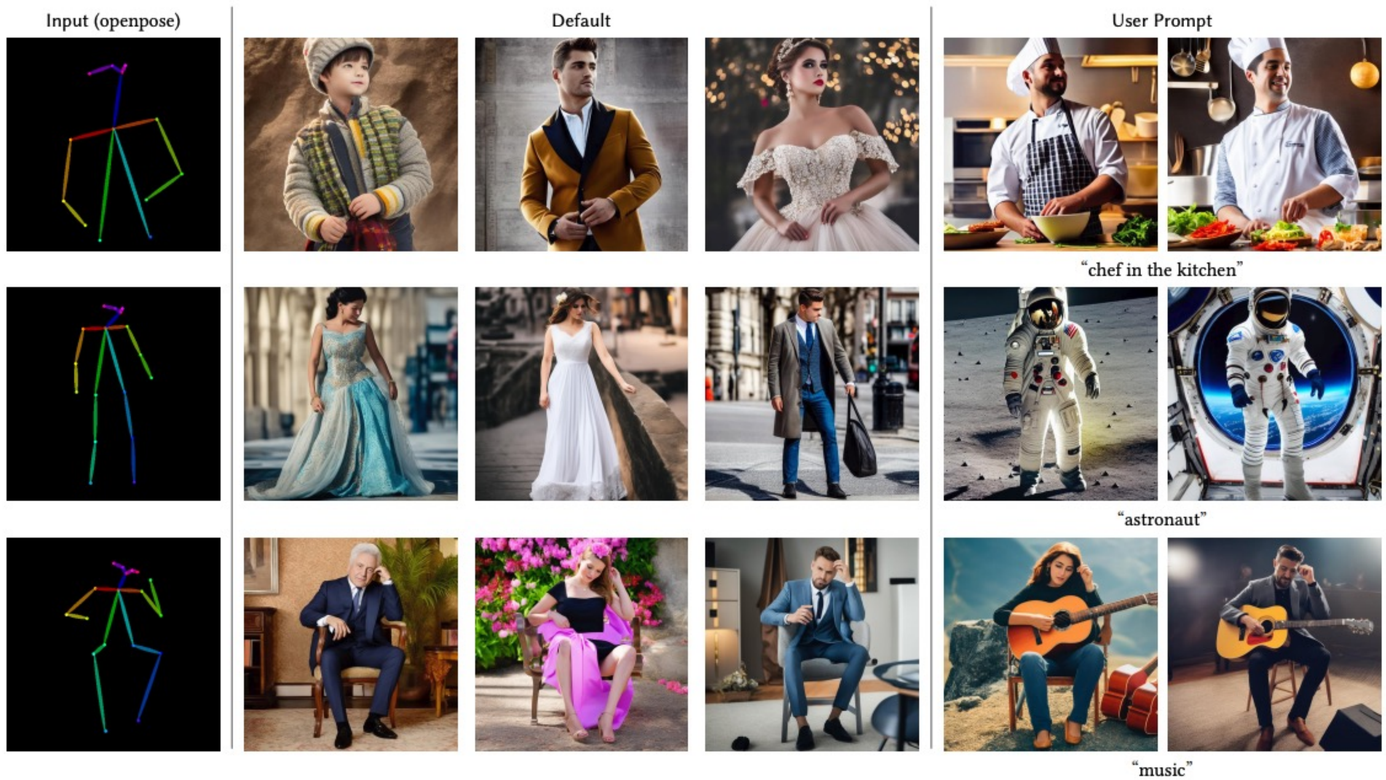
- Generation conditioned on edges, keypoints, etc.



L. Zhang and M. Agrawala. [Adding Conditional Control to Text-to-Image Diffusion Models.](#) ICCV 2023

Generative modeling tasks

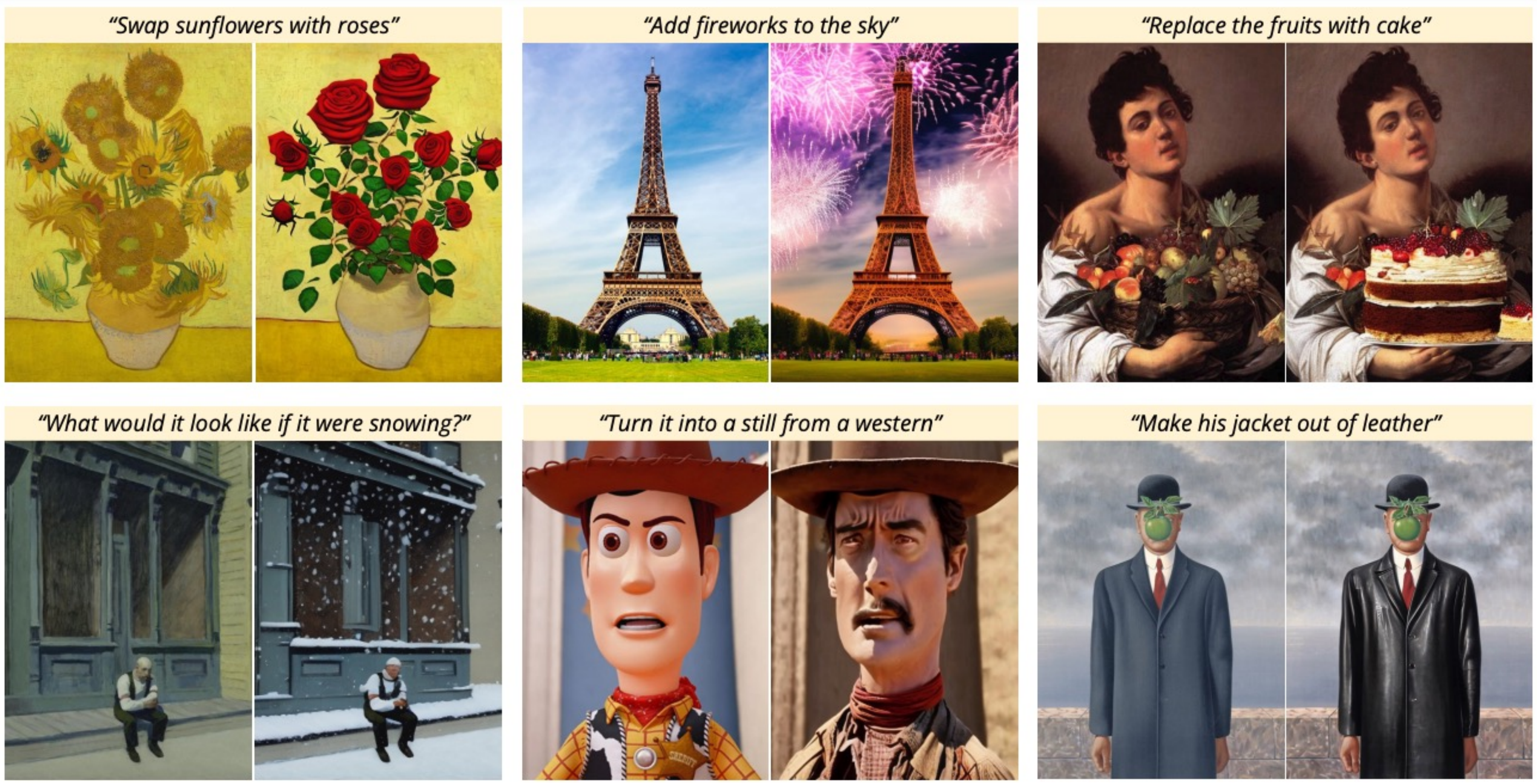
- Generation conditioned on edges, keypoints, etc.



L. Zhang and M. Agrawala. [Adding Conditional Control to Text-to-Image Diffusion Models](). ICCV 2023

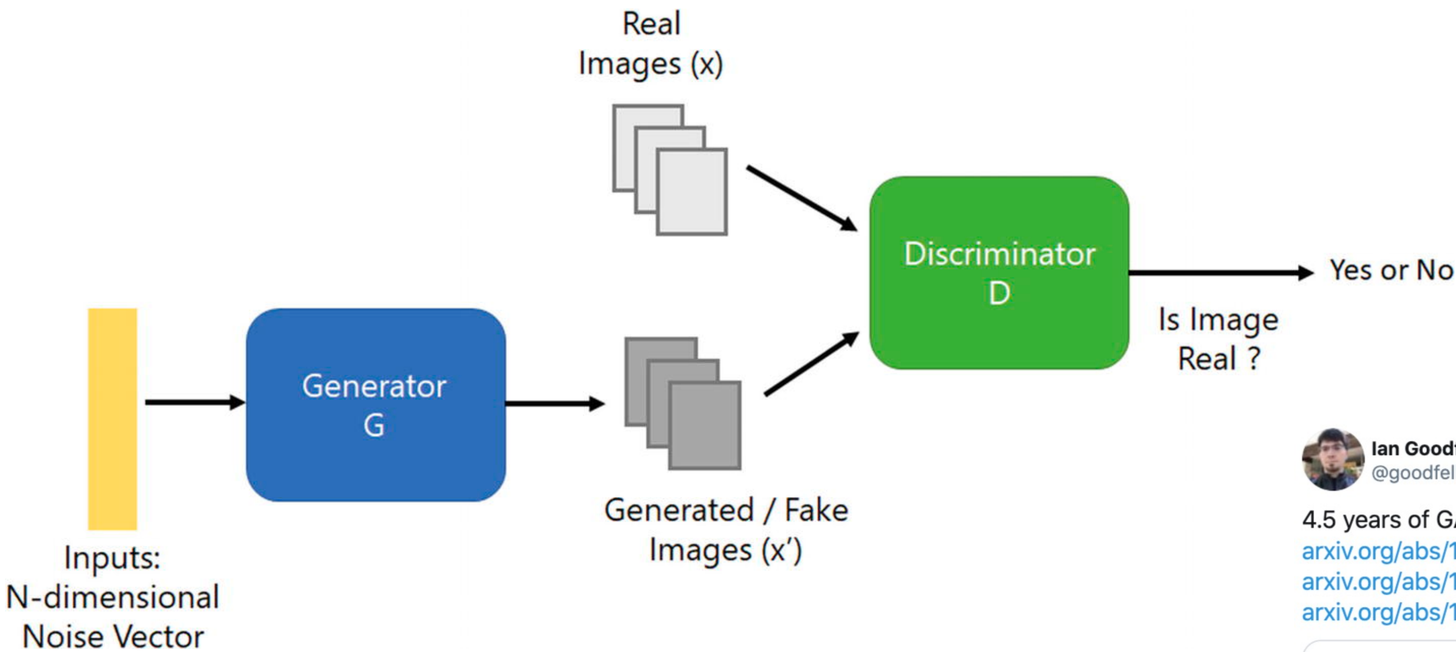
Generative modeling tasks

- Image editing or image-to-image translation



T. Brooks, A. Holynski, and A. Efros. [InstructPix2Pix: Learning to Follow Image Editing Instructions.]() CVPR 2023

Before diffusion models: Generative adversarial networks



[Image source]()

Diffusion models: Outline

- Denoising diffusion probabilistic models (DDPMs)
- Guidance: classifier-based and classifier-free
- First large-scale diffusion model: DALL-E 2
- Latent diffusion models
- Diffusion transformers and flow matching

Denoising diffusion probabilistic models (DDPMs)

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decompression scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at `https://github.com/hojonathanho/diffusion`.

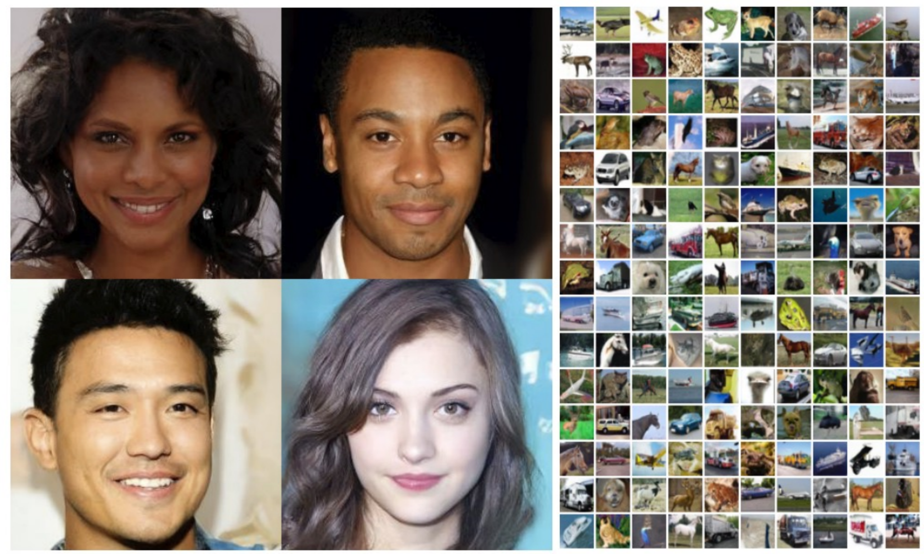
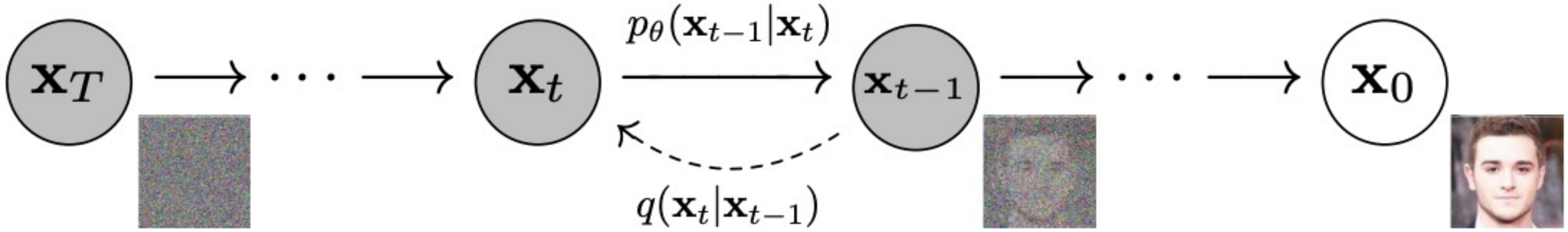


Figure 1: Generated samples on CelebA-HQ 256 $\times$ 256 (left) and unconditional CIFAR10 (right)

J. Ho et al. [Denoising diffusion probabilistic models](). NeurIPS 2020

DDPMs: Basic idea



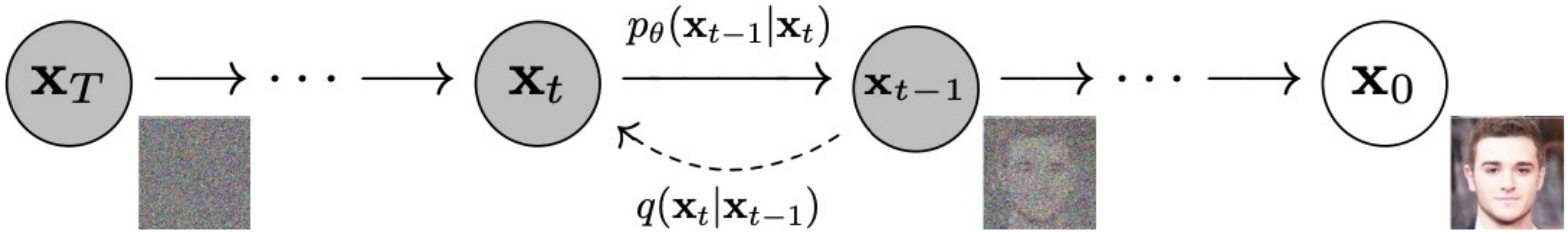
Unconditional CIFAR10 sample generation

J. Ho et al. Denoising diffusion probabilistic models. NeurIPS 2020

Blog introduction: https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

CVPR 2022 tutorial

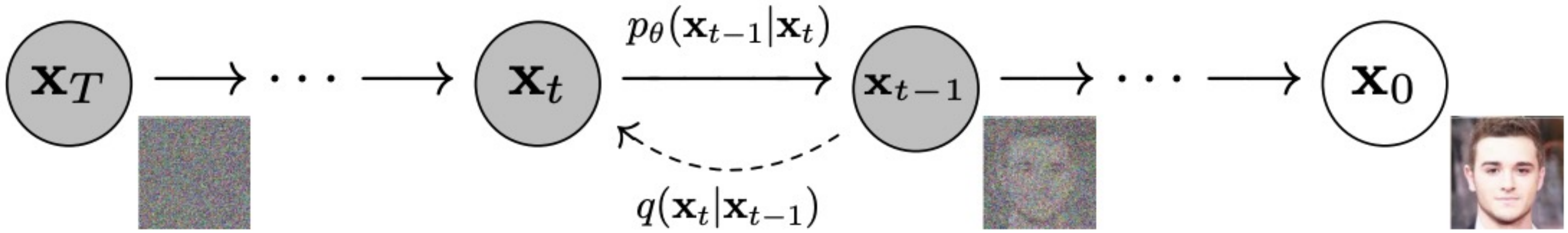
DDPMs: Basic idea



- *Forward process* q turns images into Gaussian noise
- *Reverse process* p turns noise into images
- Provided the increments of t are small enough, $p_\theta(x_{t-1}|x_t)$ is Gaussian and we can train a neural network to estimate the mean of x_{t-1} given x_t

J. Ho et al. Denoising diffusion probabilistic models. NeurIPS 2020

DDPMs: Basic idea



Algorithm 1 Training

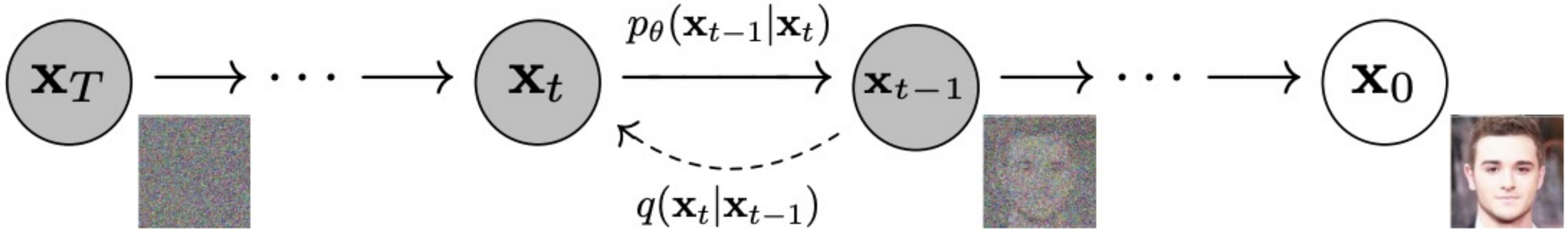
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on

$$\boxed{\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2}$$

6: **until** converged

J. Ho et al. [Denoising diffusion probabilistic models](). NeurIPS 2020

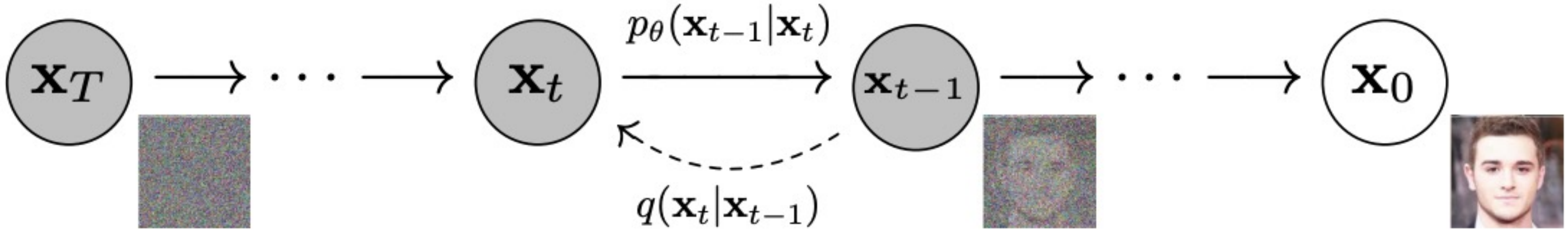
DDPMs: Basic idea



$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\underbrace{\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}}_{\text{Noisy image } \mathbf{x}_t}, t\big) \right\|^2$$

J. Ho et al. [Denoising diffusion probabilistic models](). NeurIPS 2020

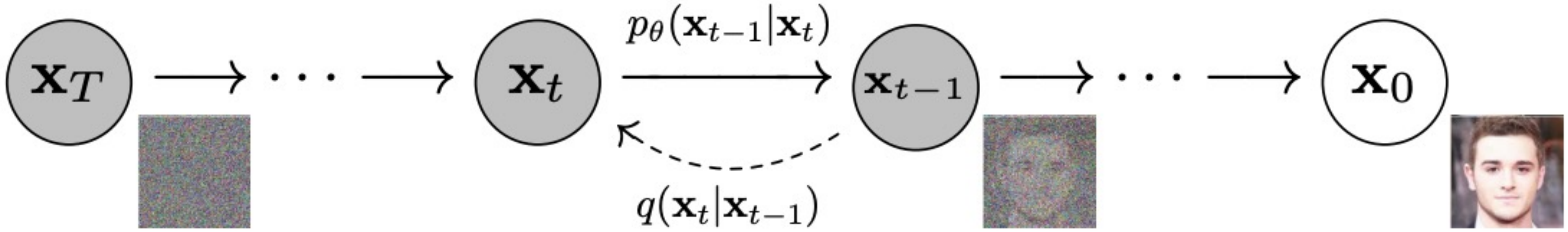
DDPMs: Basic idea



$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \underbrace{\boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t)}_{\text{Noise estimate } \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)} \right\|^2$$

J. Ho et al. Denoising diffusion probabilistic models. NeurIPS 2020

DDPMs: Basic idea

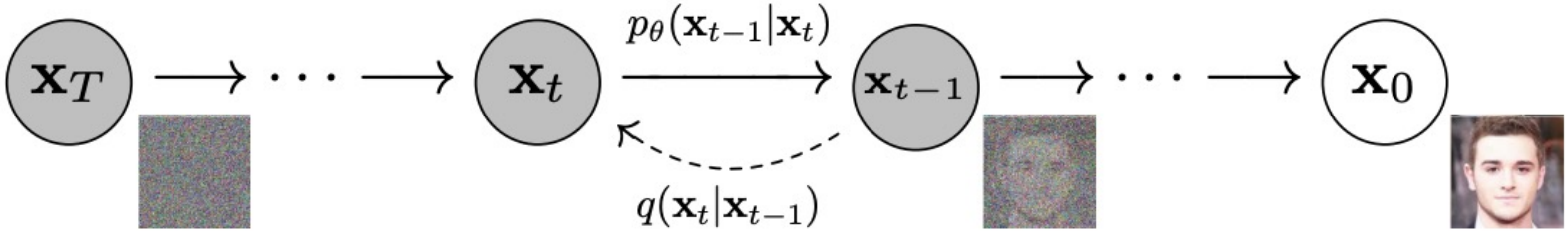


$$\nabla_\theta \left\| \underbrace{\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t)} \right\|^2$$

Difference between actual noise $\boldsymbol{\epsilon}$
and noise estimate $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$

J. Ho et al. Denoising diffusion probabilistic models. NeurIPS 2020

DDPMs: Basic idea

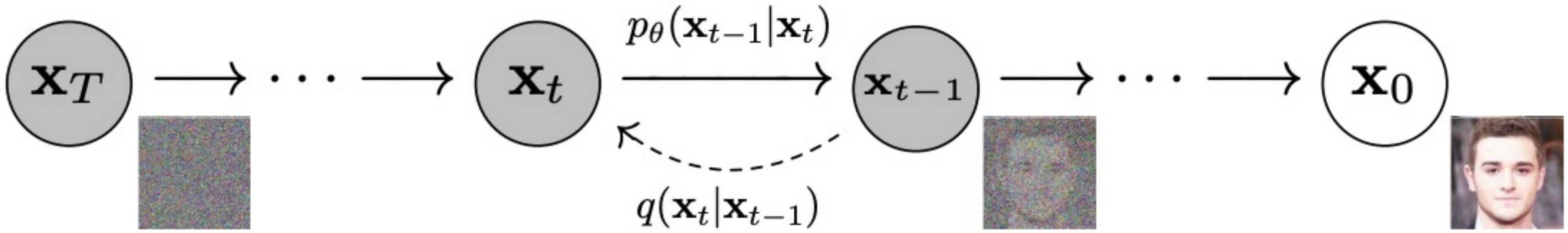


$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t\right) \right\|^2$$

Update model parameters θ to reduce the difference between actual noise $\boldsymbol{\epsilon}$ and noise estimate $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$

J. Ho et al. Denoising diffusion probabilistic models. NeurIPS 2020

DDPMs: Basic idea

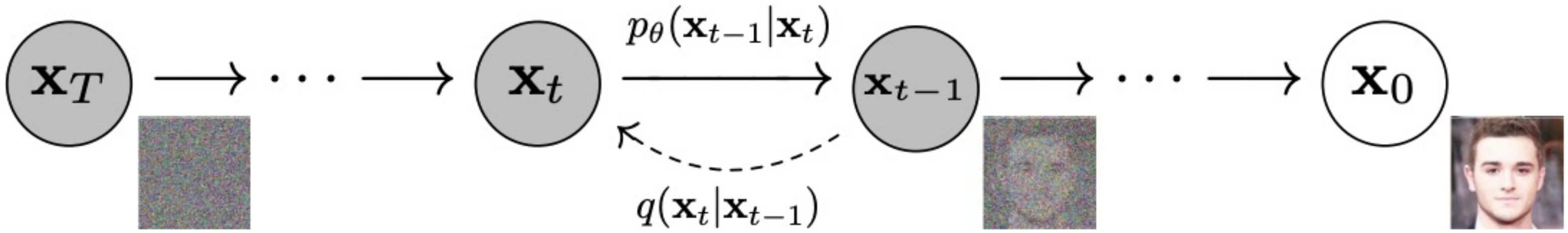


Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

J. Ho et al. Denoising diffusion probabilistic models. NeurIPS 2020

DDPMs: Basic idea



Algorithm 1 Training

repeat
 $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
 $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 Take gradient descent step on
 $$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
until converged

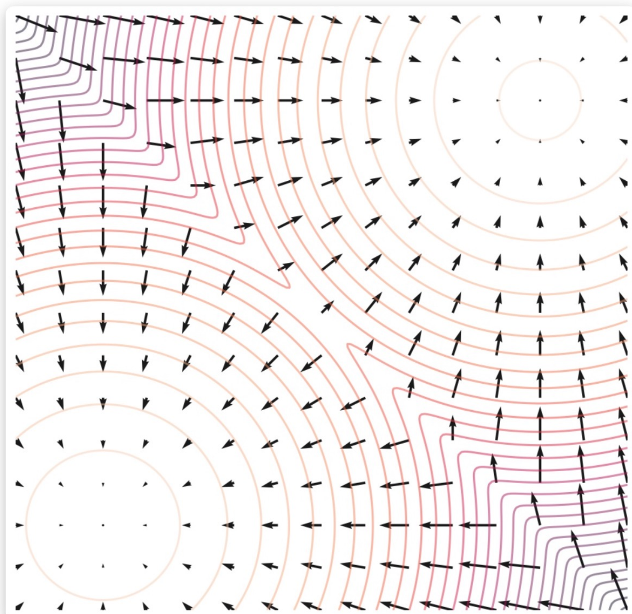
Algorithm 2 Sampling

$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = T, \ldots, 1$ **do**
 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
 $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
end for
return $\mathbf{x}_0$

J. Ho et al. Denoising diffusion probabilistic models. NeurIPS 2020

Alternate viewpoint: Score-based generative modeling

- It can be shown that $\epsilon_\theta(x_t, t) \approx -\nabla_{x_t} \log q(x_t)$, where $\nabla_{x_t} \log q(x_t)$ is the *score function* of the (noisy) data distribution

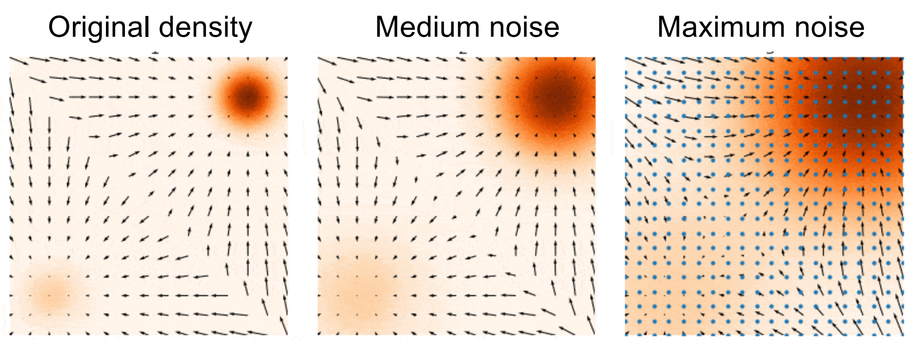


Score function (the vector field) and density function (contours) of a mixture of two Gaussians.

Y. Song and S. Ermon. [Generative Modeling by Estimating Gradients of the Data Distribution](https://yang-song.net/blog/2021/score/). NeurIPS 2019
https://yang-song.net/blog/2021/score/

Alternate viewpoint: Score-based generative modeling

- It can be shown that $\epsilon_\theta(x_t, t) \approx -\nabla_{x_t} \log q(x_t)$, where $\nabla_{x_t} \log q(x_t)$ is the *score function* of the (noisy) data distribution
- To sample from the original data density $q(x_0)$, we can use *annealed Langevin dynamics,* i.e., start by sampling from noise-perturbed versions of the data distribution and gradually reduce the amount of noise



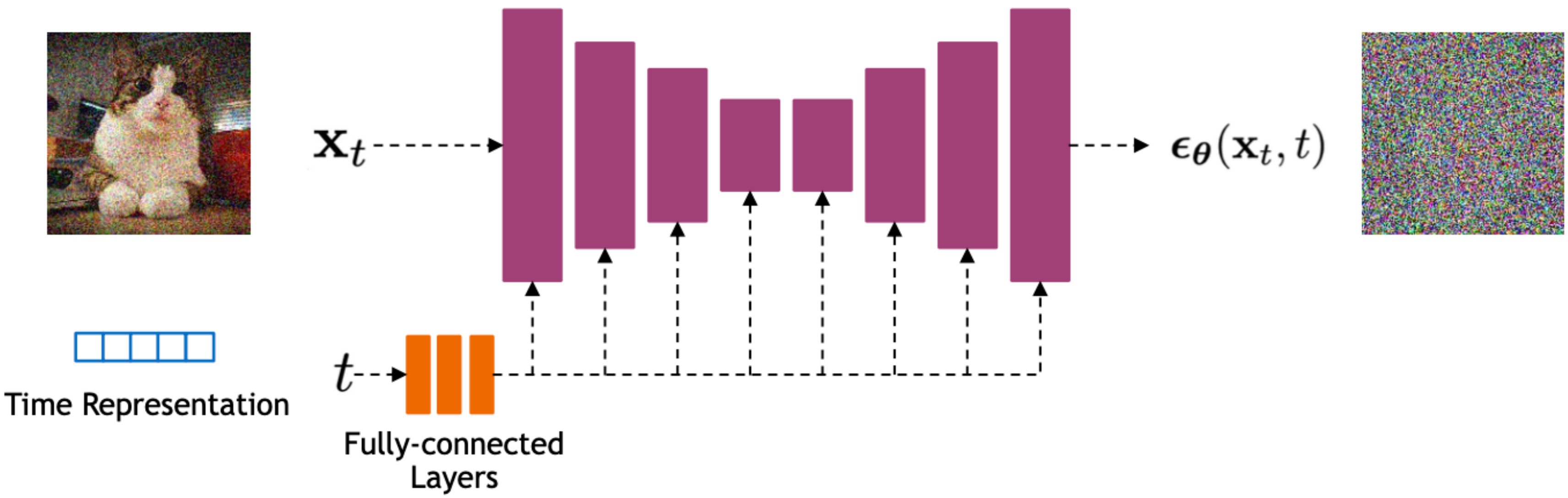
Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Y. Song and S. Ermon. Generative Modeling by Estimating Gradients of the Data Distribution. NeurIPS 2019
https://yang-song.net/blog/2021/score/

DDPMs: Implementation

- Use a U-Net architecture to represent $\epsilon_\theta(x_t, t)$
  - Bells and whistles: residual blocks, self-attention



- Encode time using sinusoidal positional embeddings or random Fourier features, fed into the U-Net using addition or adaptive normalization

Source: [CVPR 2002 DM tutorial]

Outline

- Denoising diffusion probabilistic models (DDPMs)
- Guidance: classifier-based and classifier-free

Class-conditioned DDPMs

- "We can sample with as few as 25 forward passes while maintaining FIDs comparable to BigGAN"

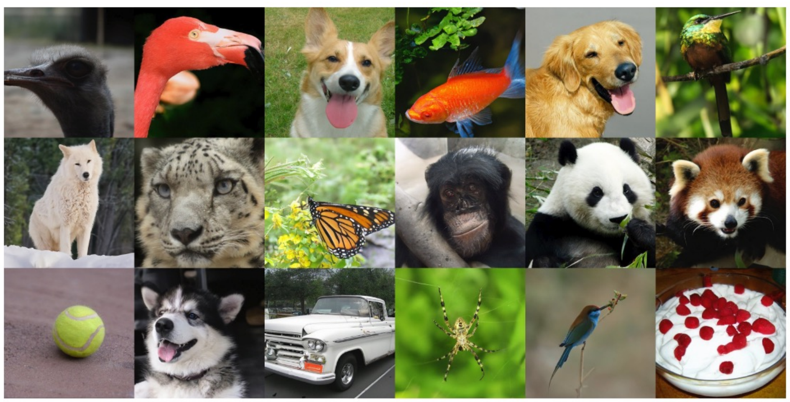


Figure 1: Selected samples from our best ImageNet 512×512 model (FID 3.85)

Abstract

We show that diffusion models can achieve image sample quality superior to the current state-of-the-art generative models. We achieve this on unconditional image synthesis by finding a better architecture through a series of ablations. For conditional image synthesis, we further improve sample quality with classifier guidance: a simple, compute-efficient method for trading off diversity for fidelity using gradients from a classifier. We achieve an FID of 2.97 on ImageNet 128×128, 4.59 on ImageNet 256×256, and 7.72 on ImageNet 512×512, and we match BigGAN-deep even with as few as 25 forward passes per sample, all while maintaining better coverage of the distribution. Finally, we find that classifier guidance combines well with upsampling diffusion models, further improving FID to 3.94 on ImageNet 256×256 and 3.85 on ImageNet 512×512. We release our code at `https://github.com/openai/guided-diffusion`.

P. Dhariwal and A. Nichol. Diffusion Models Beat GANs on Image Synthesis. NeurIPS 2021

Classifier guidance

- We can sample from the class-conditional density $q(x_t|c)$ with the help of a pre-trained classifier $P(c|x_t)$
- Bayes rule:

$$q(x_t|c) \propto P(c|x_t)q(x_t)$$

$$\log q(x_t|c) = \log P(c|x_t) + \log q(x_t) + \text{const.}$$

$$\underbrace{\nabla_{x_t} \log q(x_t|c)}_{\text{conditional score function}} = \underbrace{\nabla_{x_t} \log P(c|x_t)}_{\text{obtained from classifier output}} + \underbrace{\nabla_{x_t} \log q(x_t)}_{\text{unconditional score function (pre-trained)}}$$

- To sample from class c, steer sample in the modified direction $\nabla_{x_t}[\log q(x_t) + \gamma \log P(c|x_t)]$

Classifier-free guidance

- Instead of training a stand-alone classifier, get an "implicit classifier" by jointly training a conditional and unconditional diffusion model
- Bayes rule: $q(x_t|c) \propto P(c|x_t)q(x_t)$ or $P(c|x_t) \propto q(x_t|c)/q(x_t)$
- We can represent both $q(x_t|c)$ and $q(x_t)$ using the same network, trained by dropping out c with some probability (corresponding to the unconditional case)

J. Ho and T. Salimans. Classifier-Free Diffusion Guidance. arXIv 2021

Conditional diffusion model

- Inject class label $\mathbf{c}$ as a one-hot vector, set the vector to all zeros to drop out conditioning information

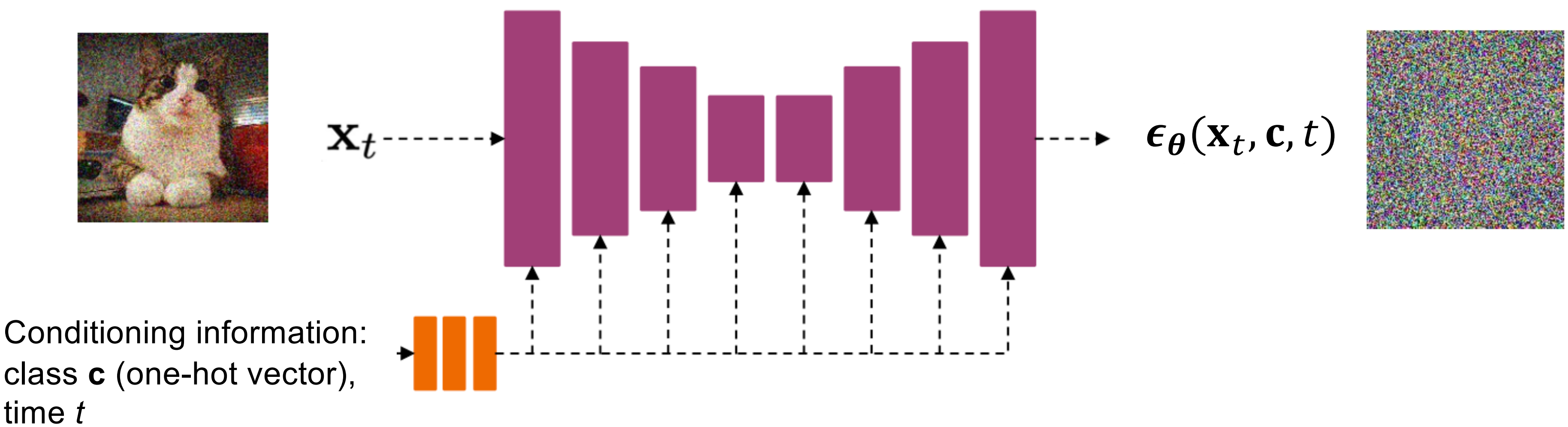


Image source

Classifier-free guidance

- Instead of training a stand-alone classifier, get an "implicit classifier" by jointly training a conditional and unconditional diffusion model
- Bayes rule: $q(x_t|c) \propto P(c|x_t)q(x_t)$ or $P(c|x_t) \propto q(x_t|c)/q(x_t)$
- We can represent both $q(x_t|c)$ and $q(x_t)$ using the same network, trained by dropping out c with some probability (corresponding to the unconditional case)
- The modified score function of this "implicit classifier" is

$$\nabla_{x_t}[\log q(x_t) + \gamma \log P(c|x_t)]$$
$$= \nabla_{x_t}[\log q(x_t) + \gamma(\log q(x_t|c) - \log q(x_t))]$$

Steer sample away from the unconditional distribution in the direction of the conditional one

J. Ho and T. Salimans. Classifier-Free Diffusion Guidance. arXIv 2021

Classifier-free guidance

Figure 1: Classifier-free guidance on the malamute class for a 64x64 ImageNet diffusion model. Left to right: increasing amounts of classifier-free guidance, starting from non-guided samples on the left.

J. Ho and T. Salimans. Classifier-Free Diffusion Guidance. arXIv 2021

Text-guided diffusion

- Instead of a class label, $\mathbf{c}$ can be an encoded text prompt

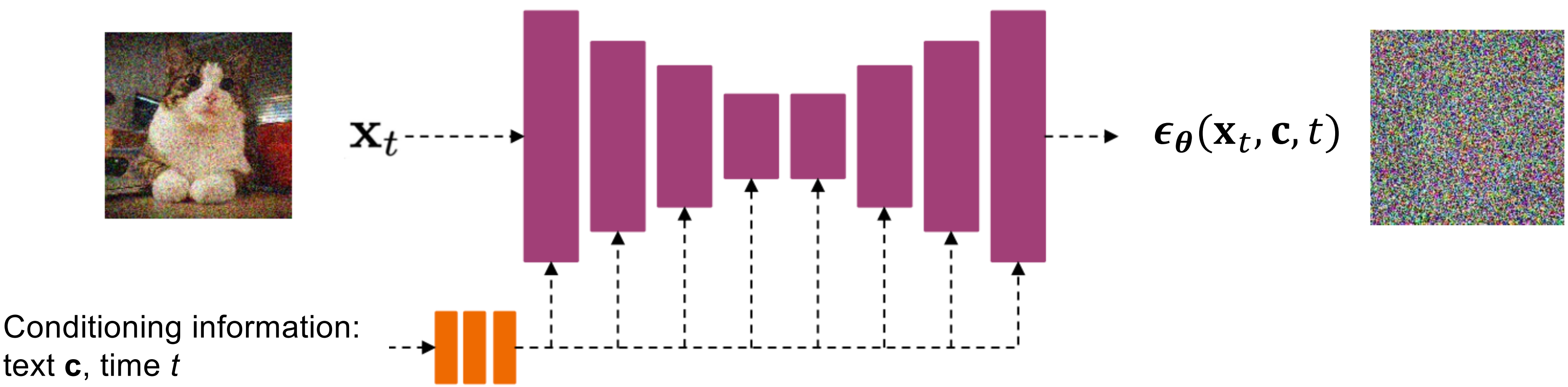
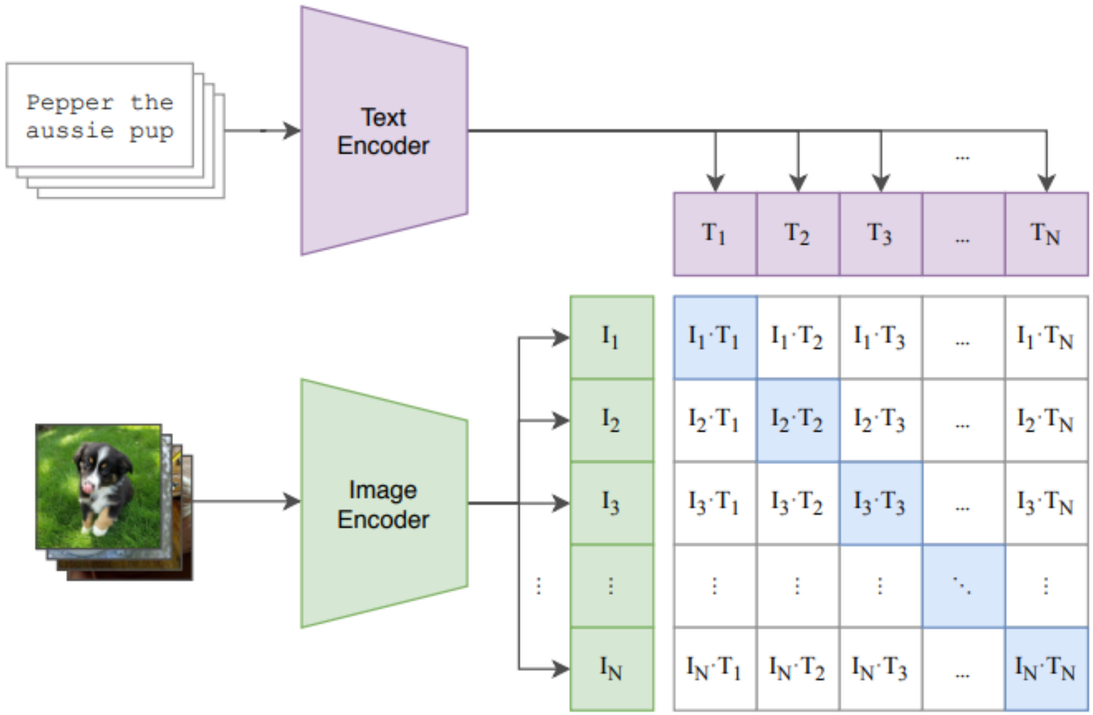


Image source

Text-guided diffusion

- Instead of a class label, $\mathbf{c}$ can be an encoded text prompt
- Classifier-free guidance works the same way as before, by training both conditional and unconditional models using text dropout
- For text guidance, need a differentiable score measuring the similarity of the current image x_t and text prompt c

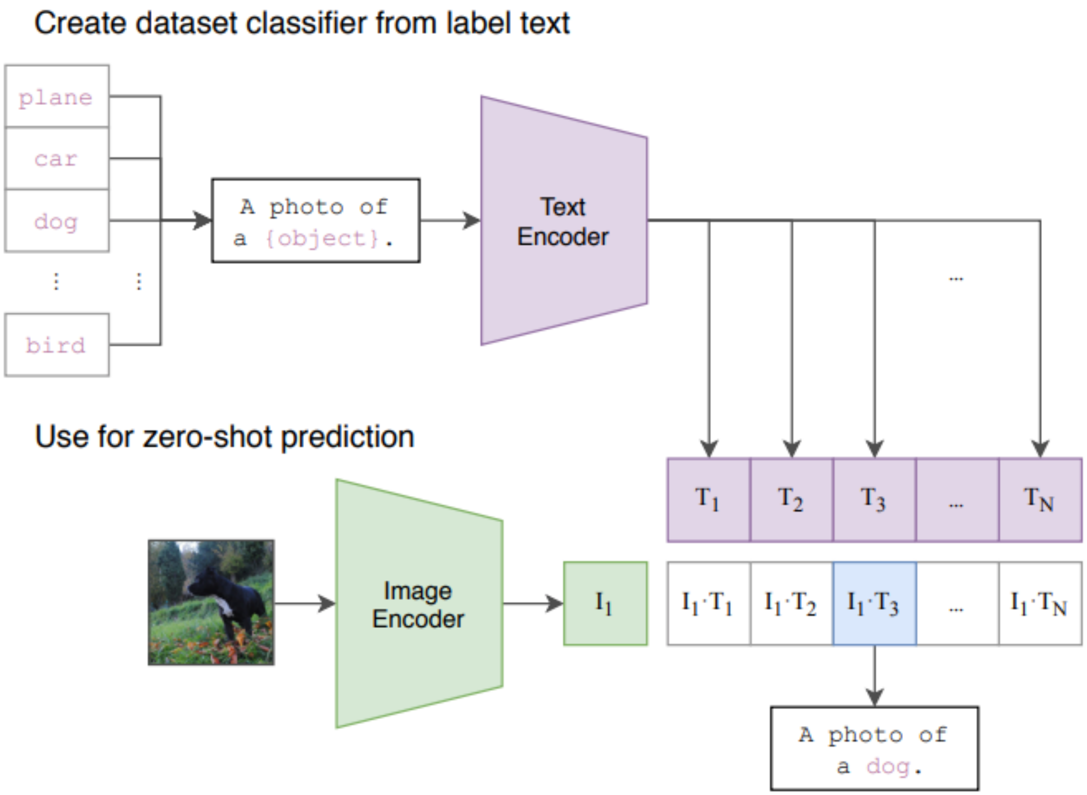
CLIP: Contrastive language-image pre-training



Contrastive objective: in a batch of N image-text pairs, classify each text string to the correct image and vice versa

A. Radford et al., [Learning Transferable Visual Models From Natural Language Supervision](), ICML 2021
[https://openai.com/blog/clip/]()

CLIP: Contrastive language-image pre-training



Original applications: image-to-text retrieval, *zero-shot* classification

A. Radford et al., [Learning Transferable Visual Models From Natural Language Supervision](), ICML 2021
[https://openai.com/blog/clip/]()

Text-guided diffusion

- Instead of a class label, $\mathbf{c}$ can be an encoded text prompt
- Classifier-free guidance works the same way as before, by training both conditional and unconditional models using text dropout
- CLIP guidance: steer samples in the direction of $\nabla_{x_t}\mathrm{CLIP}(x_t, c)$
  - Note: both classifier and CLIP must be *noise-aware* (trained on noised images)

Text-guided diffusion

- Instead of a class label, $\mathbf{c}$ can be an encoded text prompt
  - Many diffusion models inject text both as a global encoding into every layer, and a per-token encoding with cross-attention

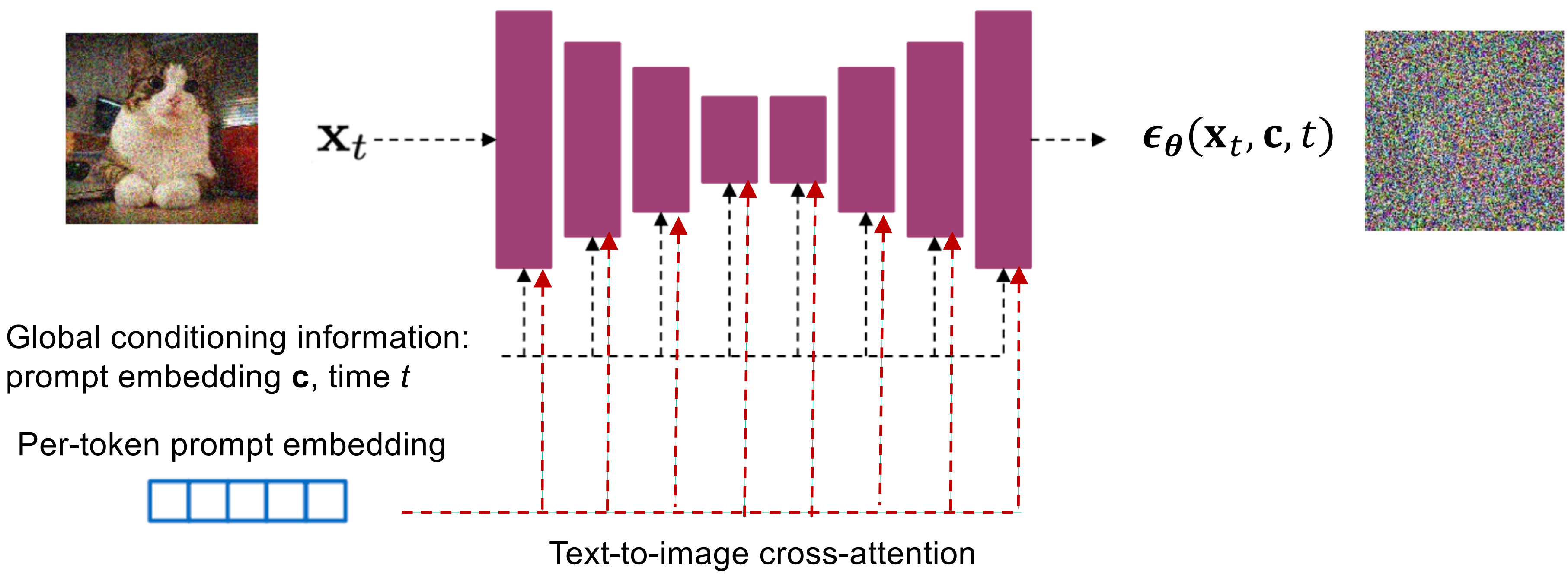


Image source

Text-to-image cross-attention

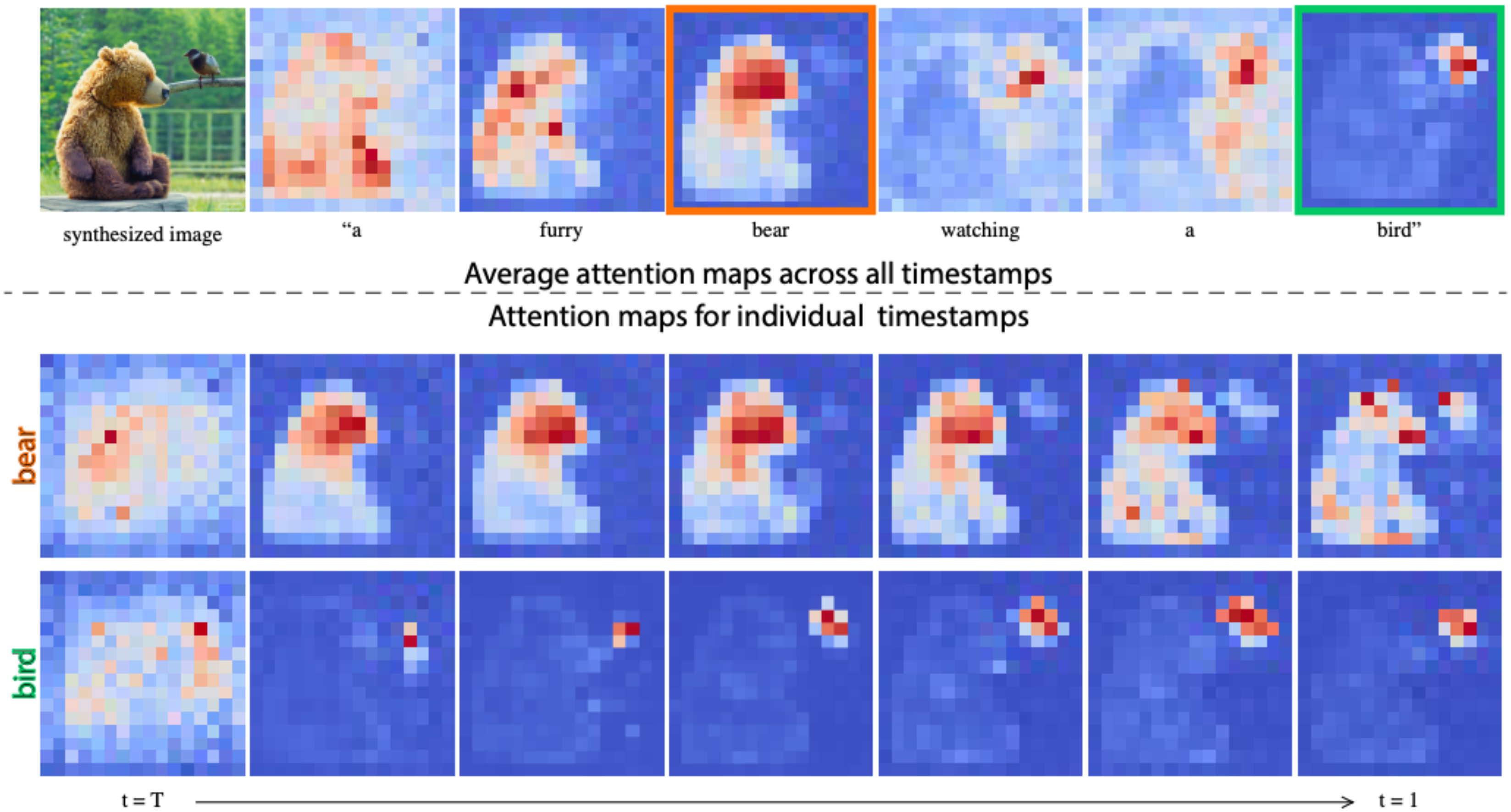


Image source

Outline

- Denoising diffusion probabilistic models (DDPMs)
- Guidance: classifier-based and classifier-free
- First large-scale model: DALL-E 2

DALL-E 2

vibrant portrait painting of Salvador Dalí with a robotic half face | a shiba inu wearing a beret and black turtleneck | a close up of a handpalm with leaves growing from it

an espresso machine that makes coffee from human souls, artstation | panda mad scientist mixing sparkling chemicals, artstation | a corgi's head depicted as an explosion of a nebula

A. Ramesh et al. [Hierarchical text-conditional image generation with CLIP latents.](#) arXiv 2022

DALL-E 2: Results

Figure 19: Random samples from unCLIP for prompt "A close up of a handpalm with leaves growing from it."

Figure 18: Random samples from unCLIP for prompt "Vibrant portrait painting of Salvador Dali with a robotic half face"

DALL-E 2: Results

Figure 3: Variations of an input image by encoding with CLIP and then decoding with a diffusion model. The variations preserve both semantic information like presence of a clock in the painting and the overlapping strokes in the logo, as well as stylistic elements like the surrealism in the painting and the color gradients in the logo, while varying the non-essential details.

DALL-E 2: Limitations

Figure 15: Reconstructions from the decoder for difficult binding problems. We find that the reconstructions mix up objects and attributes. In the first two examples, the model mixes up the color of two objects. In the rightmost example, the model does not reliably reconstruct the relative size of two objects.

Outline

- Denoising diffusion probabilistic models (DDPMs)
- Guidance: classifier-based and classifier-free
- First large-scale diffusion model: DALL-E 2
- Latent diffusion models

Latent diffusion model (basis of Stable Diffusion)

- Key idea: train a separate *encoder* and *decoder* to convert images to and from a lower-dimensional latent space, run conditional diffusion model in latent space

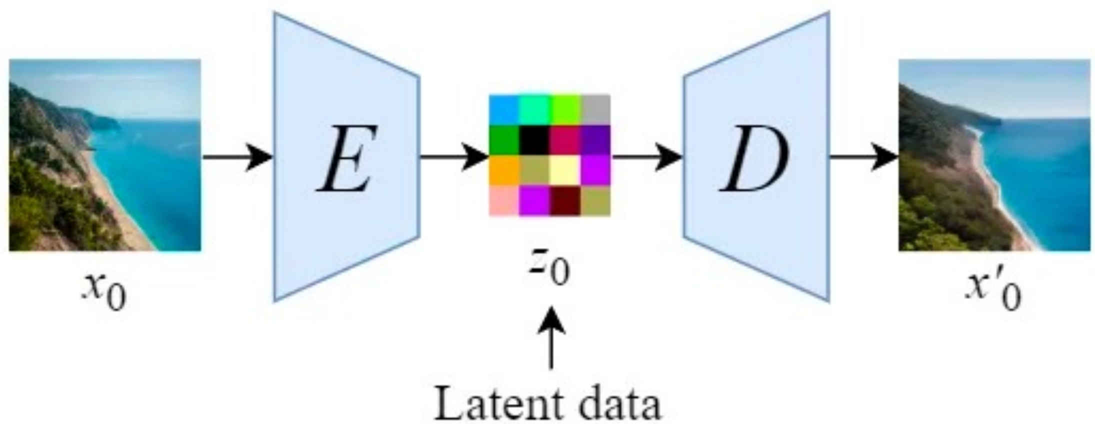
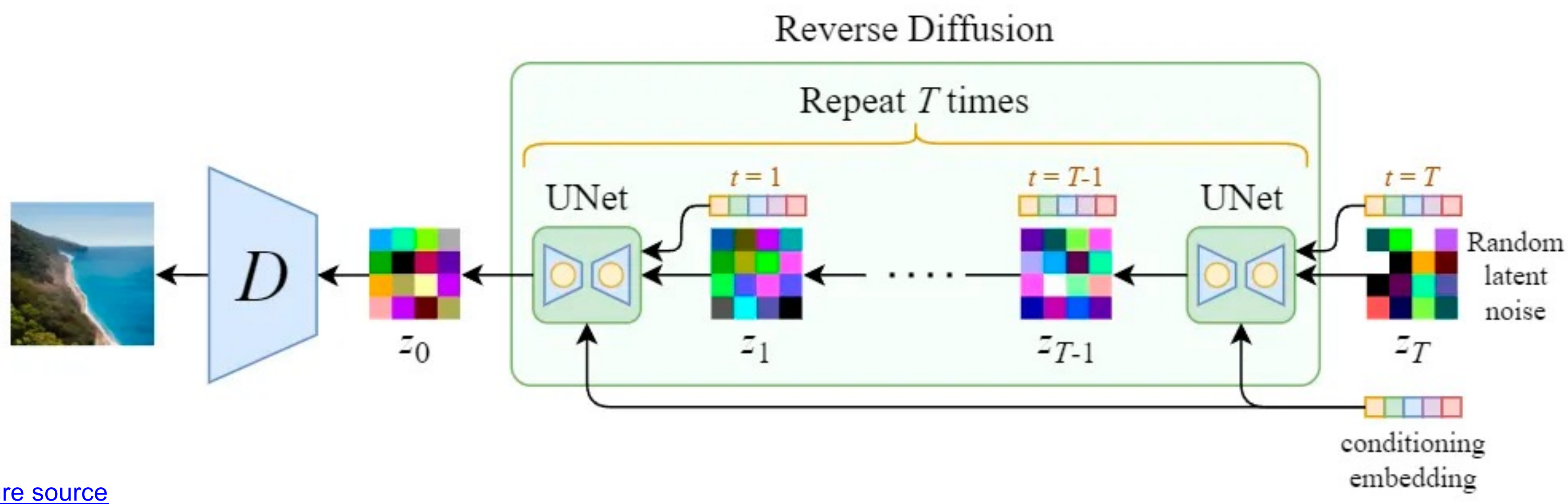


Figure source

R. Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. CVPR 2022

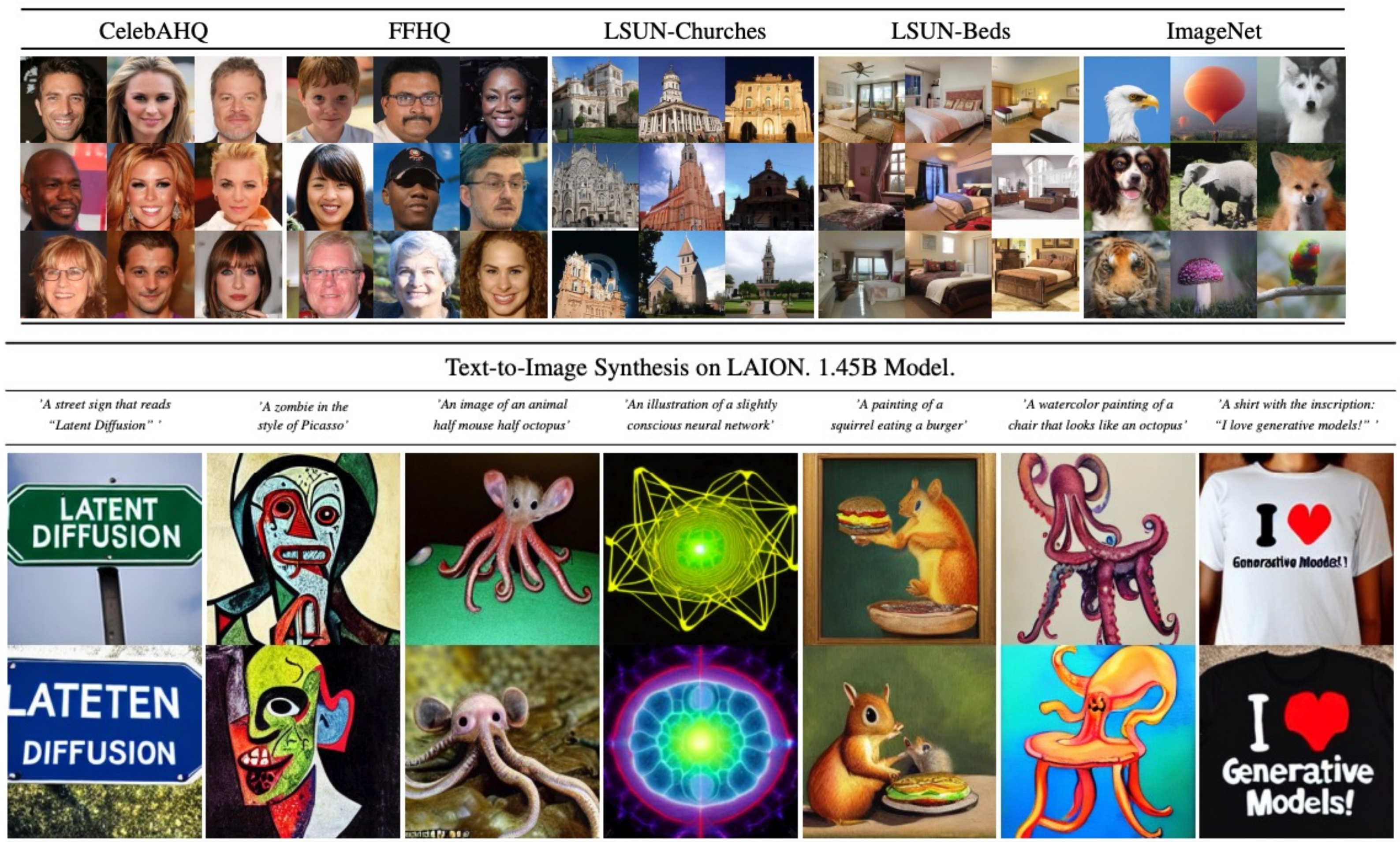
Latent diffusion model (basis of Stable Diffusion)

- Key idea: train a separate *encoder* and *decoder* to convert images to and from a lower-dimensional latent space, run conditional diffusion model in latent space

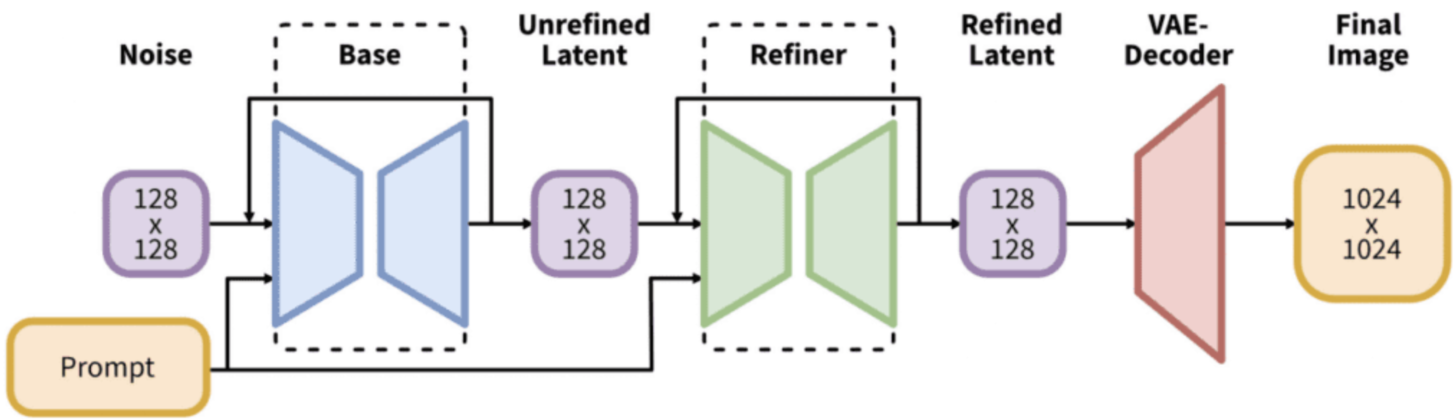


[Figure source](#)

Latent diffusion model (basis of Stable Diffusion)



Improved SDXL pipeline



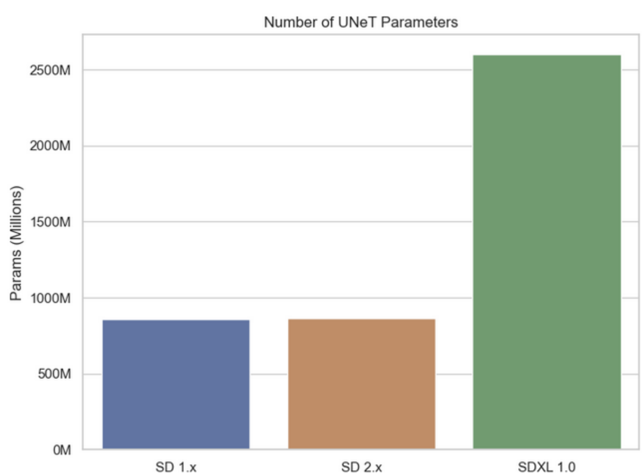
We present *Stable Diffusion XL* (*SDXL*), a latent diffusion model for text-to-image synthesis. Compared to previous versions of *Stable Diffusion*, *SDXL* leverages a three times larger UNet backbone, achieved by significantly increasing the number of attention blocks and including a second text encoder. Further, we design multiple novel conditioning schemes and train *SDXL* on multiple aspect ratios. To ensure highest quality results, we also introduce a *refinement model* which is used to improve the visual fidelity of samples generated by *SDXL* using a post-hoc *image-to-image* technique. We demonstrate that *SDXL* improves dramatically over previous versions of *Stable Diffusion* and achieves results competitive with those of black-box state-of-the-art image generators such as Midjourney (Holz, 2023).

D. Podell et al. SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis. ICLR 2024

Improved SDXL pipeline

Table 1: Comparison of *SDXL* and older *Stable Diffusion* models.

Model	*SDXL*	SD 1.4/1.5	SD 2.0/2.1
# of UNet params	2.6B	860M	865M
Transformer blocks	[0, 2, 10]	[1, 1, 1, 1]	[1, 1, 1, 1]
Channel mult.	[1, 2, 4]	[1, 2, 4, 4]	[1, 2, 4, 4]
Text encoder	CLIP ViT-L & OpenCLIP ViT-bigG	CLIP ViT-L	OpenCLIP ViT-H
Context dim.	2048	768	1024
Pooled text emb.	OpenCLIP ViT-bigG	N/A	N/A



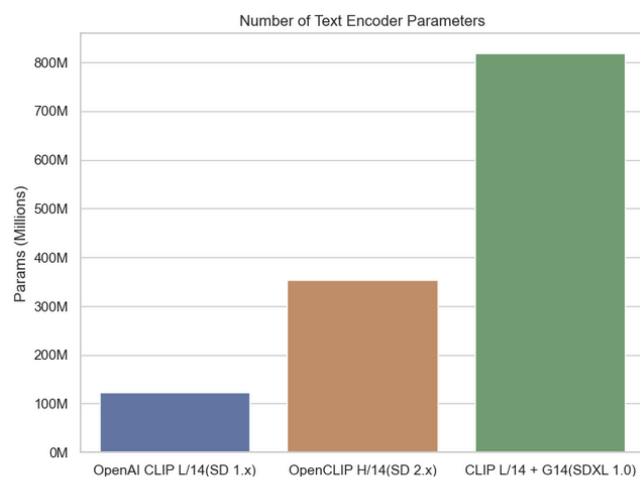


Figure source

D. Podell et al. SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis. ICLR 2024

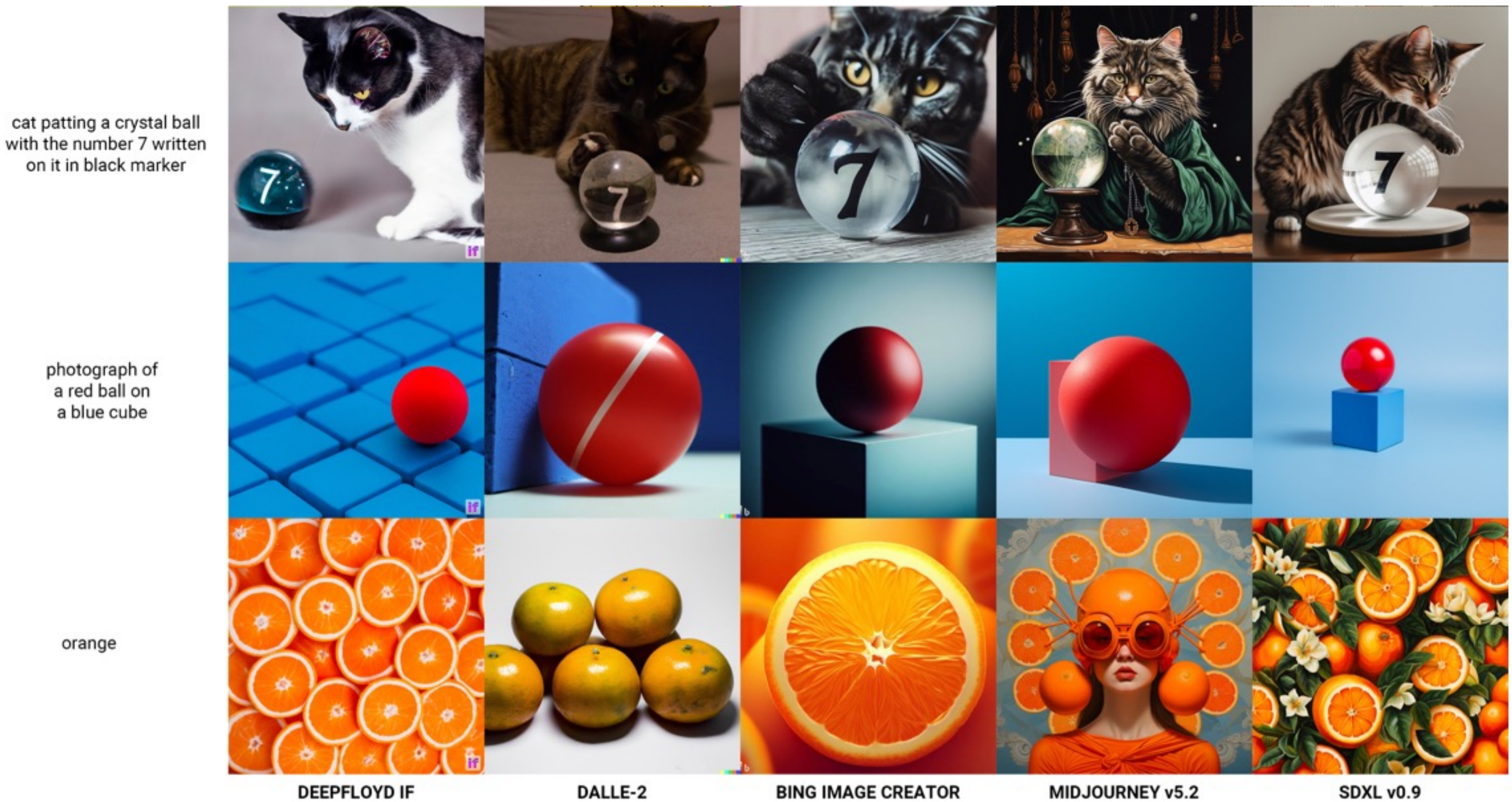
SDXL: Results

'A propaganda poster depicting a cat dressed as french emperor napoleon holding a piece of cheese.'

'a close-up of a fire spitting dragon, cinematic shot.'

Figure 4: Comparison of the output of *SDXL* with previous versions of *Stable Diffusion*. For each prompt, we show 3 random samples of the respective model for 50 steps of the DDIM sampler [46] and cfg-scale 8.0 [13]. Additional samples in Fig. 14.

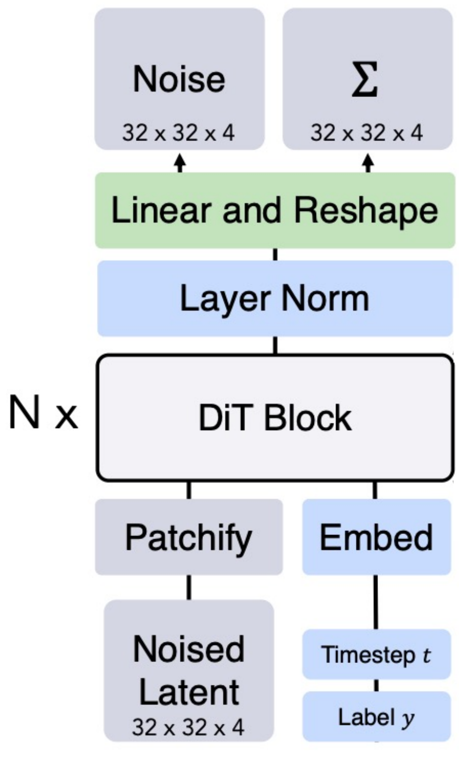
SDXL: Results



Diffusion models: Outline

- Denoising diffusion probabilistic models (DDPMs)
- Guidance: classifier-based and classifier-free
- First large-scale diffusion model: DALL-E 2
- Latent diffusion models
- Diffusion transformers and flow matching

Diffusion transformers (DiT)



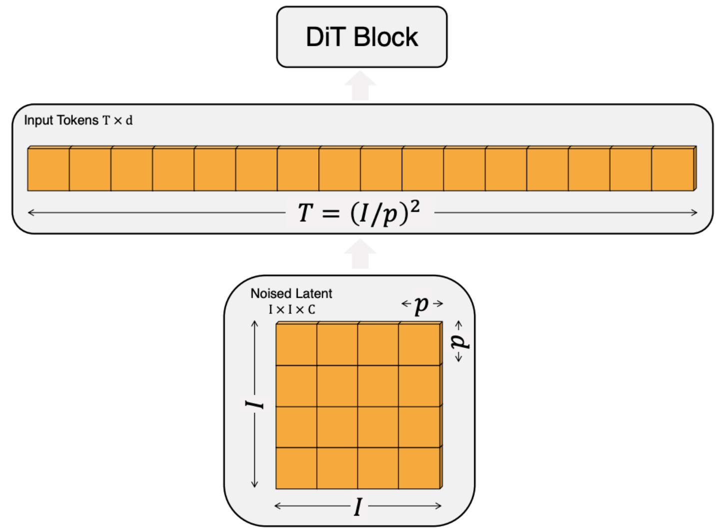


Figure 4: **Input specifications for DiT.** Given patch size $p \times p$, a spatial representation (the noised latent from the VAE) of shape $I \times I \times C$ is "patchified" into a sequence of length $T = (I/p)^2$ with hidden dimension d. A smaller patch size p results in a longer sequence length and thus more Gflops.

W. Peebles and S. Xie. Scalable Diffusion Models with Transformers. ICCV 2023

Diffusion transformers (DiT)

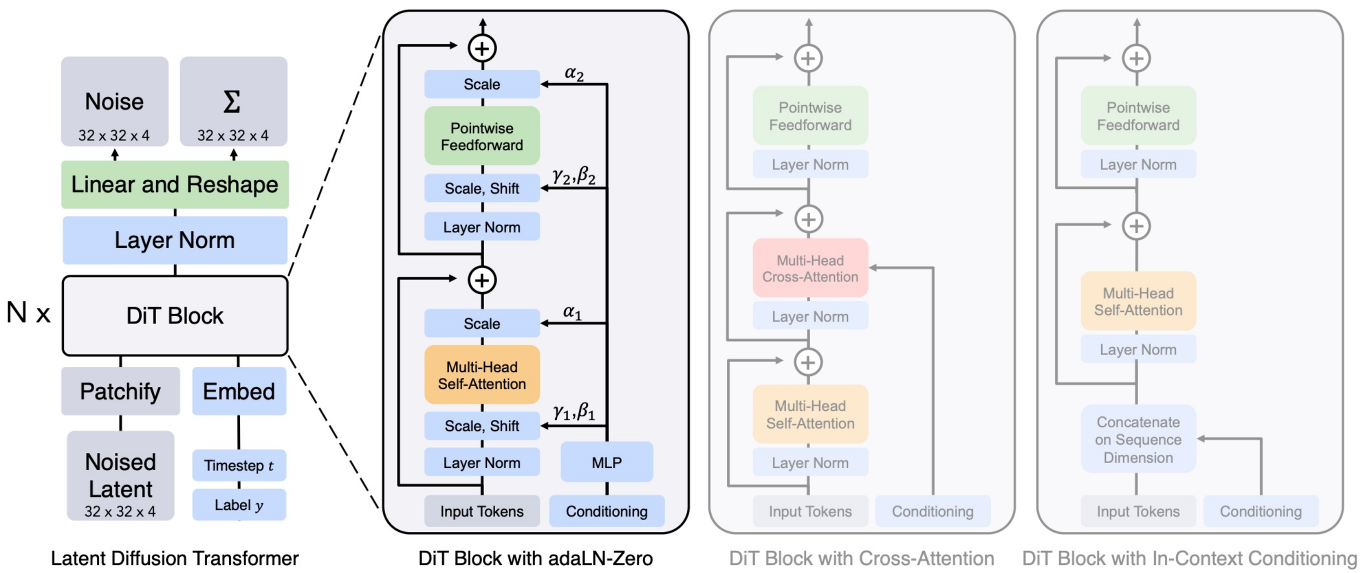


Figure 3: **The Diffusion Transformer (DiT) architecture.** *Left:* We train conditional latent DiT models. The input latent is decomposed into patches and processed by several DiT blocks. *Right:* Details of our DiT blocks. We experiment with variants of standard transformer blocks that incorporate conditioning via adaptive layer norm, cross-attention and extra input tokens. Adaptive layer norm works best.

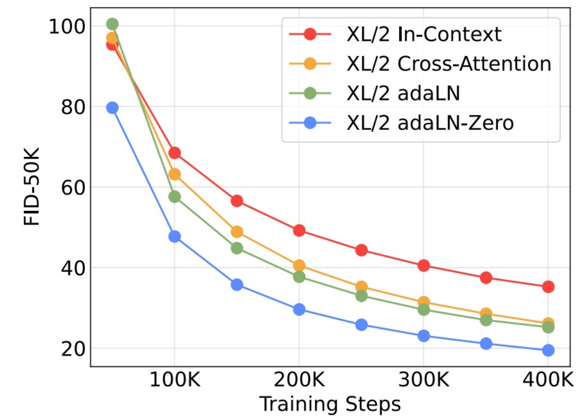
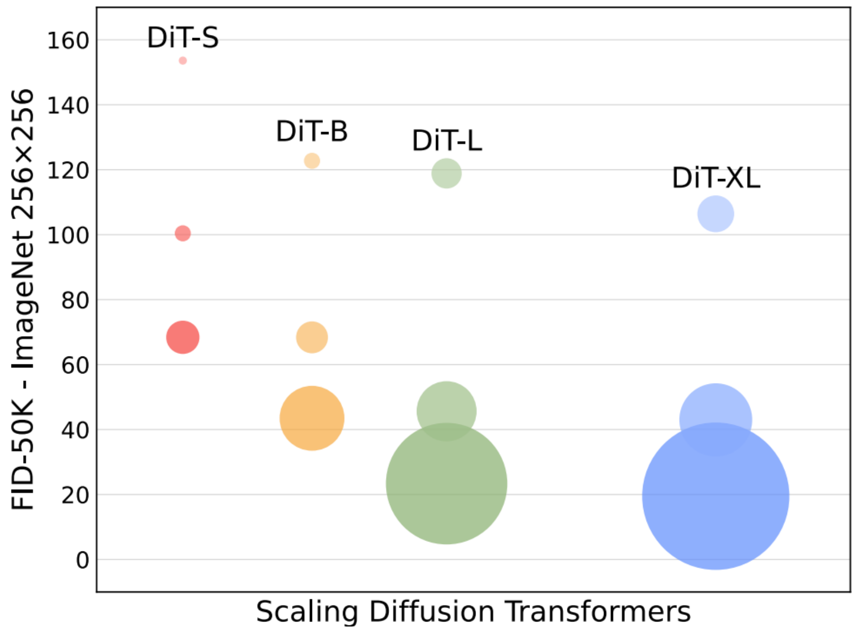


Figure 5: **Comparing different conditioning strategies.** adaLN-Zero outperforms cross-attention and in-context conditioning at all stages of training.

W. Peebles and S. Xie. [Scalable Diffusion Models with Transformers](). ICCV 2023

Diffusion transformers (DiT)



Model	Layers N	Hidden size d	Heads	Gflops (I=32, p=4)
DiT-S	12	384	6	1.4
DiT-B	12	768	12	5.6
DiT-L	24	1024	16	19.7
DiT-XL	28	1152	16	29.1

Table 1: **Details of DiT models.** We follow ViT [10] model configurations for the Small (S), Base (B) and Large (L) variants; we also introduce an XLarge (XL) config as our largest model.

W. Peebles and S. Xie. Scalable Diffusion Models with Transformers. ICCV 2023

Diffusion transformers (DiT)

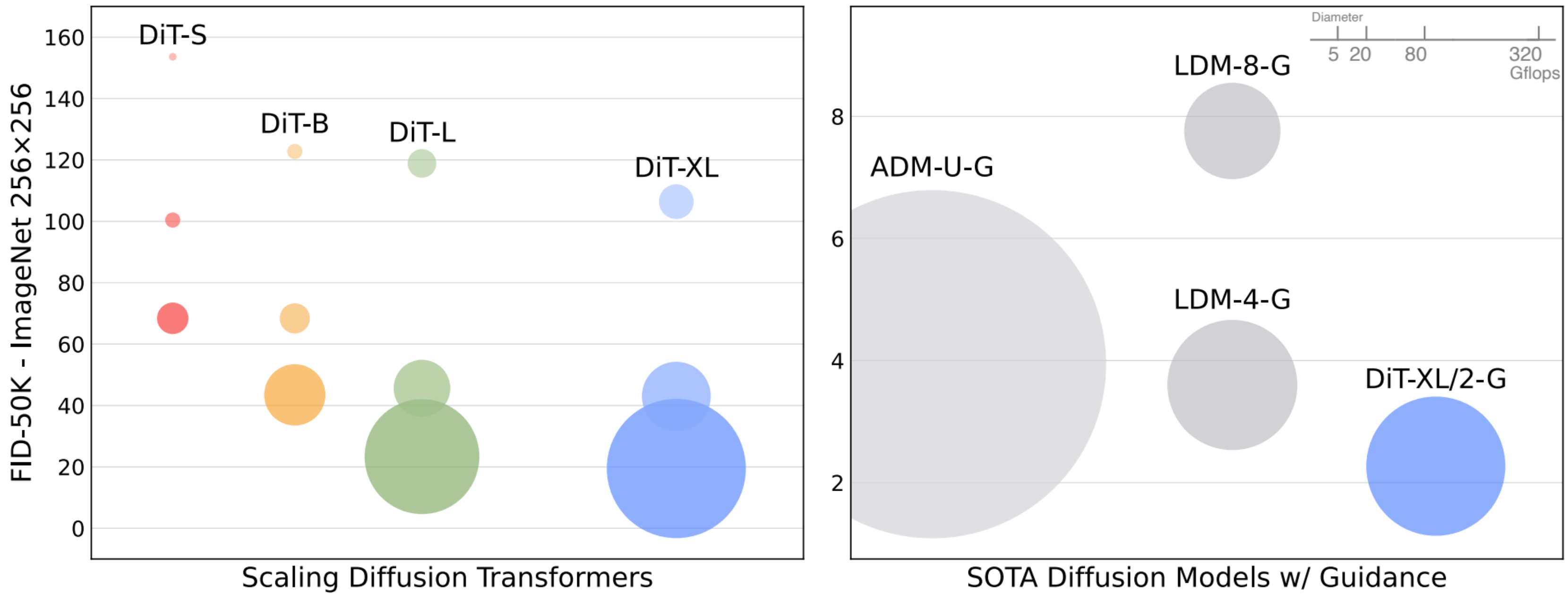
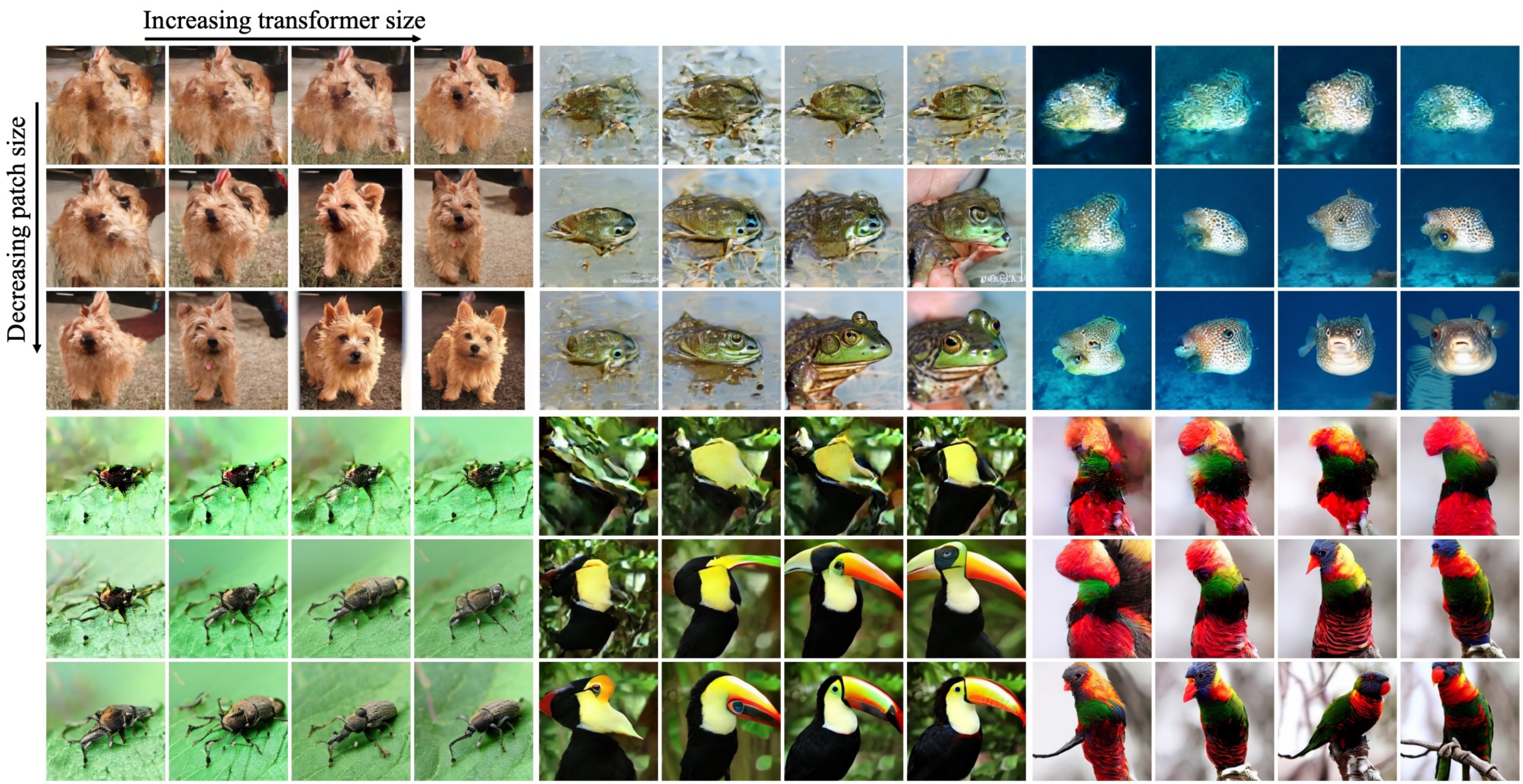


Figure 2: **ImageNet generation with Diffusion Transformers (DiTs).** Bubble area indicates the flops of the diffusion model. *Left:* FID-50K (lower is better) of our DiT models at 400K training iterations. Performance steadily improves in FID as model flops increase. *Right:* Our best model, DiT-XL/2, is compute-efficient and outperforms all prior U-Net-based diffusion models, like ADM and LDM.

W. Peebles and S. Xie. [Scalable Diffusion Models with Transformers](). ICCV 2023

Diffusion transformers (DiT)

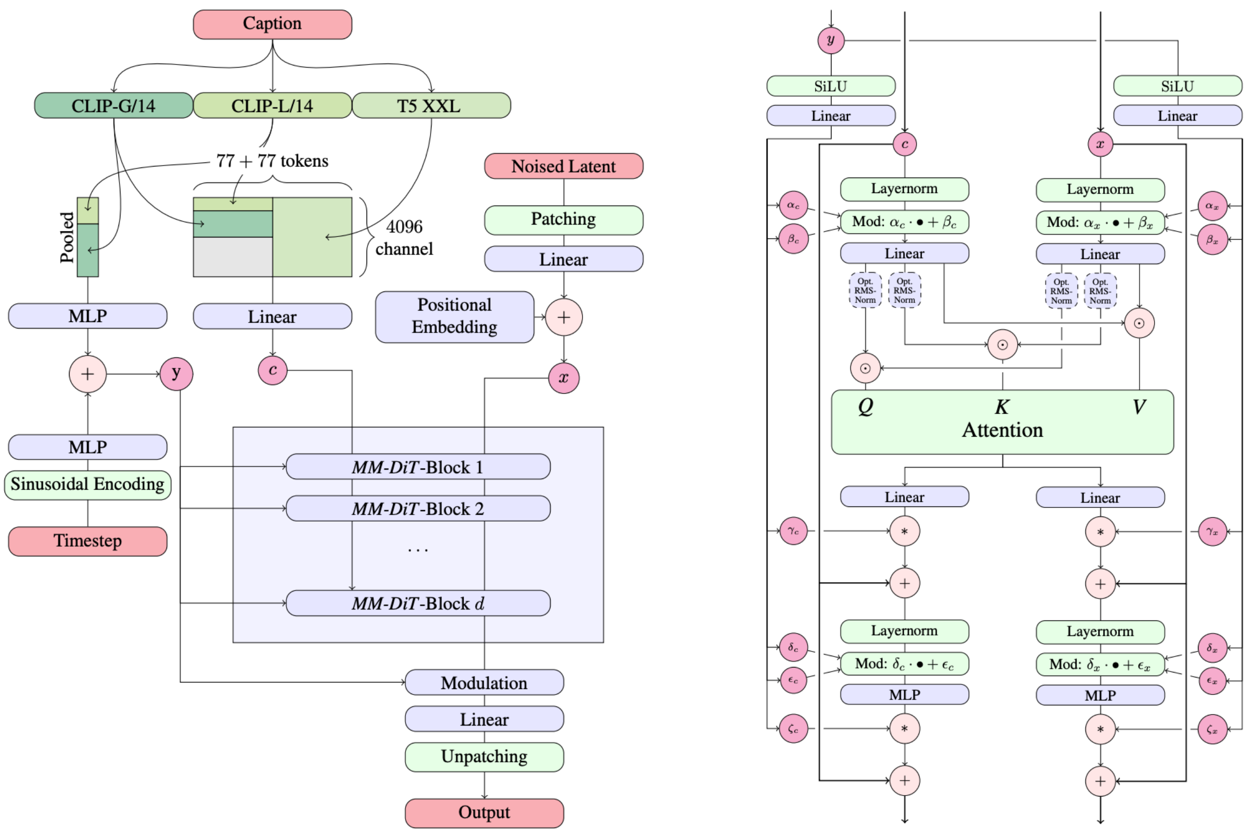


SD3: Diffusion transformers and rectified flow

Figure 1. High-resolution samples from our 8B rectified flow model, showcasing its capabilities in typography, precise prompt following and spatial reasoning, attention to fine details, and high image quality across a wide variety of styles.

P. Esser et al. [Scaling Rectified Flow Transformers for High-Resolution Image Synthesis](). ICML 2024

SD3

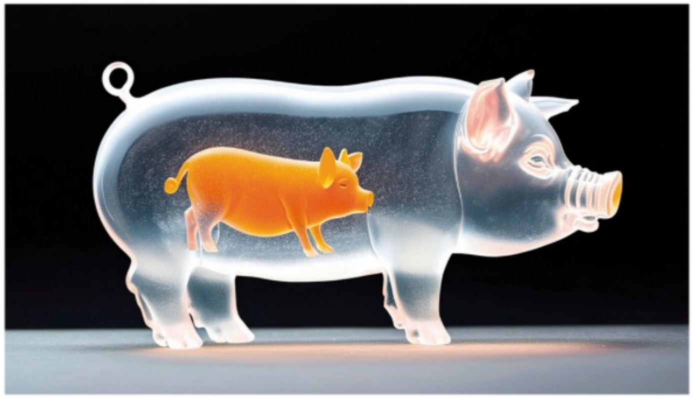


(a) Overview of all components.

(b) One *MM-DiT* block

P. Esser et al. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. ICML 2024

SD3: Results



Prompt: Translucent pig, inside is a smaller pig.

Prompt: A massive alien space ship that is shaped like a pretzel.

Prompt: A kangaroo holding a beer, wearing ski goggles and passionately singing silly songs.

Prompt: An entire universe inside a bottle sitting on the shelf at walmart on sale.

Prompt: A cheeseburger with juicy beef patties and melted cheese sits on top of a toilet that looks like a

Prompt: This dreamlike digital art captures a vibrant, kaleidoscopic bird in a lush rainforest

Prompt: A car made out of vegetables.

Prompt: Heat death of the universe line art

SD3: Results

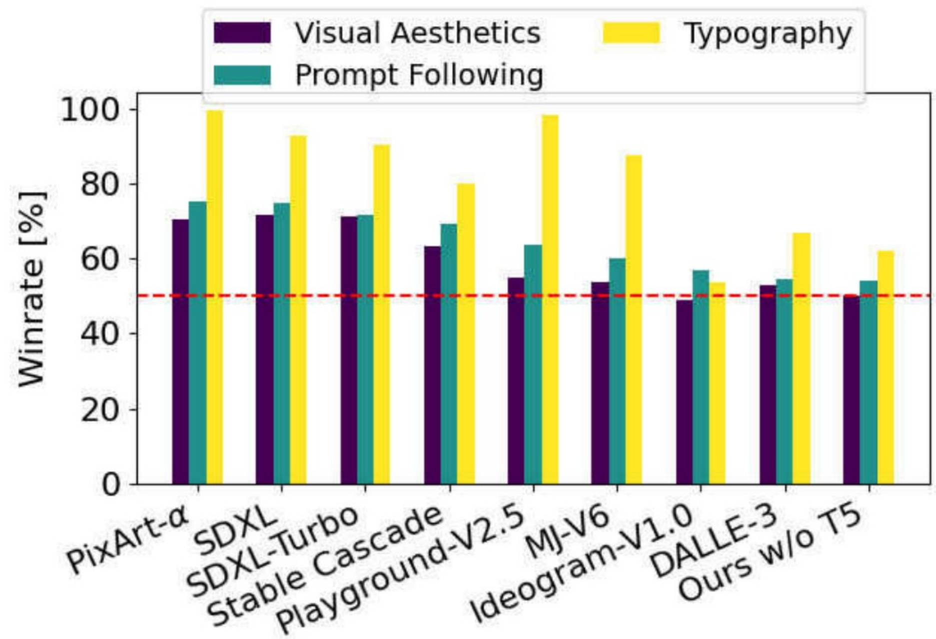


Figure 6. **Human Preference Evaluation against currrent closed and open SOTA generative image models.** Our 8B model compares favorable against current state-of-the-art text-to-image models when evaluated on the parti-prompts (Yu et al., 2022) across the categories *visual quality*, *prompt following* and *typography generation*.